



Software Inspections – 7

- Practical Issues for Success

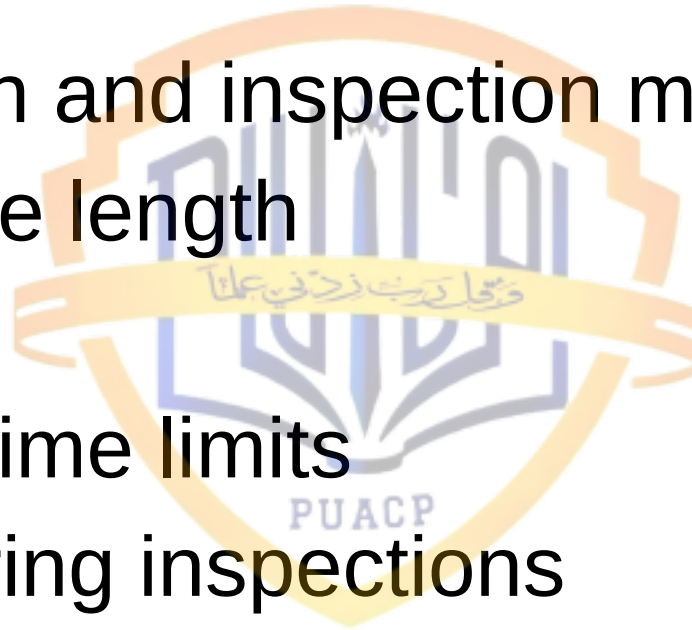
Lecture # 21

Today's Lecture

- We understand the need and importance of software inspections
- We also know that software inspections are effective and efficient mechanism for removal of software defects
- Today' we'll discuss some practical issues for success related to software inspections

Timing Issues

- Preparation and inspection meetings
- Reasonable length
- Subsets
- Enforcing time limits
- Breaks during inspections
- Scheduled times



When is an Inspection Finished?

■ Planned

- Includes any time during the project's life cycle where the schedule has been defined for a required inspection; it concludes with the inspection meeting start

■ Performed

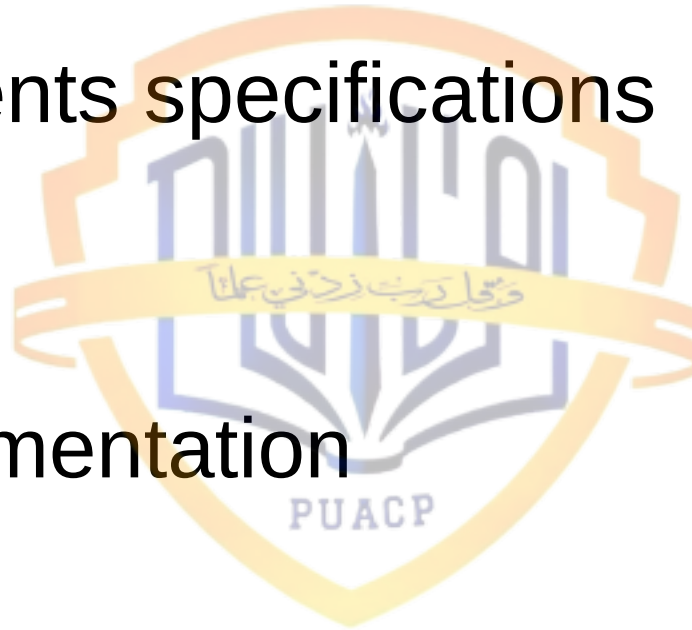
- Includes all times from the start of the inspection meeting through rework

■ Closed

- Is only after follow-up when closure has been achieved and signed-off

Best Place to Start First

- Requirements specifications
- Design
- Code
- User documentation



Can Some Work Products Not be Inspected?

- Yes, but the decision requires data that demonstrates minimal risk and good data requires time in practice
- For safety-critical or life-critical software, you should not take the risk lightly, if at all

Who are the Right Inspectors? - 1

- A domain expert will be far more effective in finding defects than a novice. Experts, however, are not always available when we want them
- Less capable inspectors may only be able to find a certain class of defects, while experts can find deeper defects, but they all can contribute to finding defects

Who are the Right Inspectors? - 2

- The decision should be based on risk and criticality of the work product. Here criticality is not just safety-critical or life-critical situations, but work products critical to the success of the project
- Re-inspection may have to be done when experts are available, in case they were not available for the inspection meeting

Inspections Don't Make People Warm and Fuzzies

- Inspections are rarely the most exciting task for programmers, but they are necessary and useful. They should be made as comfortable as possible

Helping Programmers to Learn from Their Errors - 1

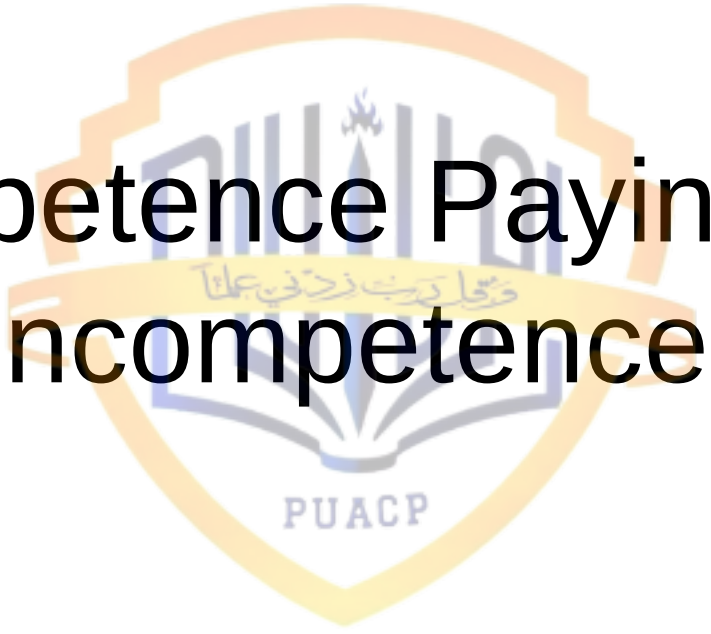
- If we allow the programmers to learn in a safe environment, they generally will learn
- Not all programmers are equal in capability, but all can contribute to the project's success
- We may have to provide additional training to some

Helping Programmers to Learn from Their Errors - 2

- As programmers learn, they will become more effective, and this will show in their work products
- They will take pride in their work and work environment



Competence Paying for Incompetence



Small Teams

- Inspections can be applied in small teams
- Such small teams do not have the formality of inspections, as we have discussed so far
- However, when another set of eyes look at the work products, defects can be identified quickly

Do We Really Need All that Data with Inspections?

- In the beginning, data may not be collected to get familiar with inspections
- However, in order to have highly effective inspections, you'll need the data and analysis on that data

Get Off to a Good Start - 1

- Not all people will accept the use of inspections initially
- Data and experience should win some of the doubters over, but holding the hands of some will be necessary
- Learn to understand the concerns of the people who are being asked to use inspections and you will be more successful

Get Off to a Good Start - 2

- The doubters will ask questions like:
 - ☐ Is the overhead worth it?
 - ☐ Does management really support this?
 - ☐ How will the data be used?
- These doubts will not be limited to practitioners. Management may have even deeper doubts and issues

Get Off to a Good Start - 3

- This is the time the inspection coordinator or SEPG must stay the course
- Responses must be given to hold the doubters' focus, and eventually the data should win them over too
- Today it is far easier with the volume of inspection data and experiences reported in the literature

Inspecting Changes - 1

- Some argue that when a change is made to a previously tested work product that it is cheaper and less labor intensive to retest rather than inspect or re-inspect
- This argument is false for three reasons

Inspecting Changes - 2

- (1) If a fix or change is made and previous test cases did not flush out the defect, then rerunning them does not prove the changes work. It only proves the change did not regress the work product. This is not verification of the change or fix! The test cases need to be changed or modified before rerunning. Otherwise they would have found the defect

Inspecting Changes - 3

- (2) When a re-inspection is performed it is not required on the whole work product. The change plus surrounding code may be sufficient to verify the change or fix

Inspecting Changes - 4

- (3) Changes and fixes have defects in them. In fact, there is data that suggests that small changes are more defective than large changes. Therefore, there is a good likelihood that a defect will be found in test, and now we are back to the very reason why inspections are better; it is cost effective to find the defect before test versus test

Inspecting Changes - 5

- Inspections are looking at change in the context of the work product and relationships it has with other work products. This means that inspectors should ask what else could be affected by the change and try to ensure that the change does not ripple into another defect

Inspecting Changes - 6

- No one ever claimed that tests would go away because of inspections; it just will be less costly. Therefore, we should expect that some regression testing should occur even after inspections are finished
- Test will validate that the product was not regressed by the fix; inspections will verify that the fix is correct

“Yes, Of Course, We Do Inspections!” - 1

- It sometimes happens that the checkmark of having done an inspection is tracked with a higher level of importance than performing an effective inspection
- For example, inspections are indicated as having been completed regardless of when they might have occurred

“Yes, Of Course, We Do Inspections!” - 2

- This results in ineffective inspections and an organization attitude that inspections are a bureaucratic quality approach



“Yes, Of Course, We Do Inspections!” - 3

- Situations where this checkmark misuse is seen
 - (1) Unit test is performed before the code inspection, and in some cases the inspection becomes a non-event
 - (2) Resources are not available for a scheduled inspection, so the work continues without an inspection and then sometime later a minimal inspection in name is held, e.g., a requirement inspection is held after or while the design is being completed

“Yes, Of Course, We Do Inspections!” - 3

- (3) Some people believe an inspection will go faster when bugs have already been taken out via test or some other approach
- One wonders why inspections are held in the first place
- Satisfy the management's decision without buying into them. People suffer

If You Find a Defect, Fix It

- Inspections find defects but producer do not fix them correctly
- Most common reasons for this sort of unprofessional behavior are:
 - The inspection data was lost
 - The author didn't understand the inspection data
 - There wasn't time in the schedule

Bureaucratic Data Collection

- Not all people will see value in collecting data on inspections



Inspections are too Formalized

- Formality is not itself a problem
- The question is whether the process is repeatedly producing positive business results
- If so, formality is justified
- If the process is onerous, inefficient, or ineffective then something should be done



Inspections Waste Time

Redundancy with the Test Processes - 1

- Inspections are intended to remove defects as early as possible. They are quality control mechanism
- Testing is intended to
 - Find the defects that leaked through the inspection process
 - Revalidate that the delivered solution satisfies the needs of the customer

Redundancy with the Test Processes - 2

- Tests are both quality control and quality assurance mechanism
- With analysis of data from the inspection and test results, both processes can be tuned to maximize efficiency and minimize redundancy, while maintaining effectiveness

Arbitrary Style Viewpoints during Inspections

- If an organization has not defined or agreed to an accepted style for specifications, design, code, or for other documents, then it is possible that inspection participants may have different viewpoints. This can lead to useless discussions
- Agreements on these issues is a prerequisite for effective and efficient inspections

Less Efficient than Debugging during Execution - 1

- (1) Some people believe that debugging within a test execution environment is faster and cheaper. The literature consistently has suggested otherwise
- Try a controlled study

Less Efficient than Debugging during Execution - 2

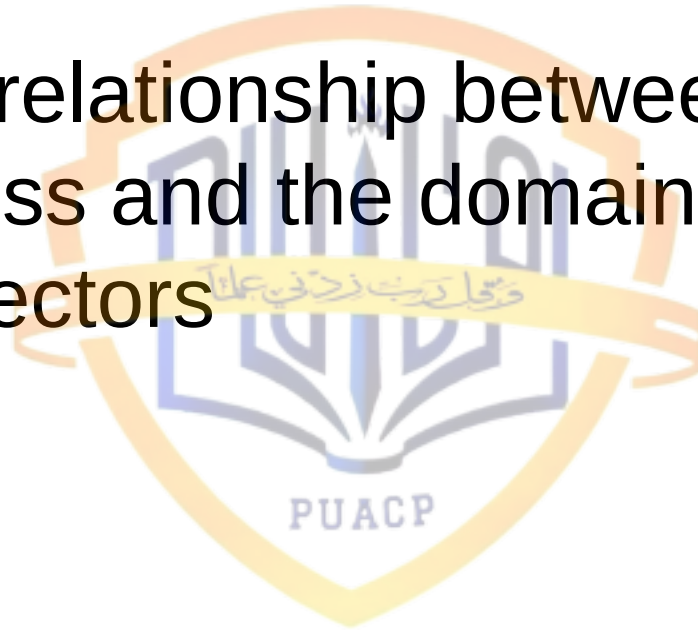
- (2) There may be languages where execution of the code during the inspection would be more beneficial. For example, the visual type of languages may be best inspected in permutation of the traditional inspection using both the code and viewing the performance of the code by looking at the screens during the execution

Design Knowledge is Required to do Code Inspections

- Absolutely, but they do not have to have as much domain knowledge as the producer
- Know code and design sufficiently
- Design and specification documents can be helpful in understanding

Only Superficial Errors are Found

- There is a relationship between effectiveness and the domain knowledge of the inspectors



Inspecting too Much

- It is possible



Performance Characteristics are Not Addressed

- If it is true, then these may not have been the focus of inspections



The “Not the Way I Would have Done It” Attitude

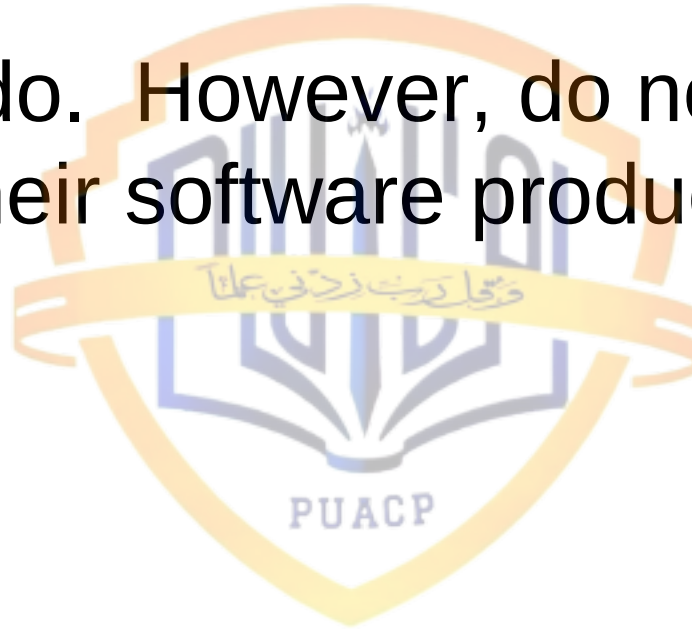
- This issue must be controlled
 - Proper and consistent training across the inspection team members
 - Documented methods, styles, and notations to be used by the producer in creating the artifact

Can We Do Less Rigorous Inspections?

- Can be done with forethought that this is a good business decision from an effectiveness perspective (i.e., very low defects)
- The informal approach is noted as such and tracked as a potential risk
- Review is made after test and customer use to learn if the decision was good

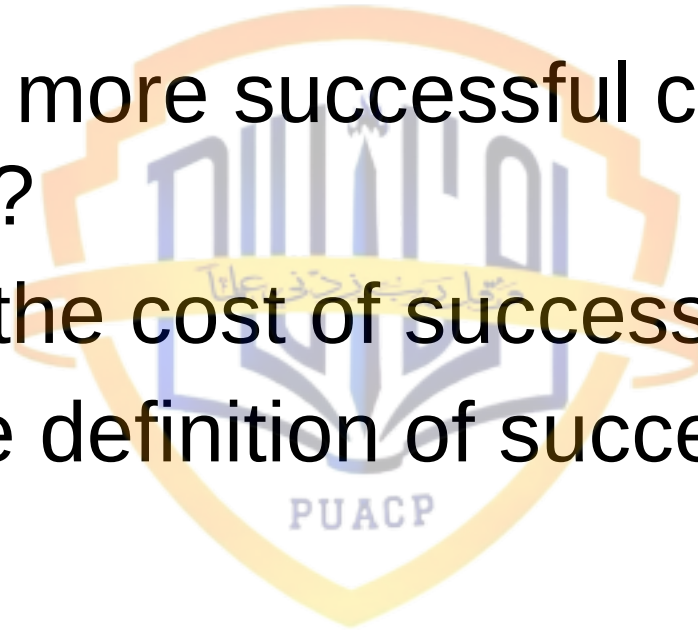
Customers Accept “Good Enough” Software Solutions

- Yes, they do. However, do not accept defect in their software products



Many Successful Companies do Not Use Inspections

- How much more successful could they have been?
- What was the cost of success?
- What is the definition of success?



Modified Code is too Costly to Inspect

- There is need to inspect the additions, deletions, and modifications plus some “surround code” to understand the context of the additions, deletions, and modifications

Some Inspectors Don't Contribute

- This will happen and can be for a number of reasons



We Tried Inspections and They Didn't Work - 1

- What does the data show?
- Did they really use inspections?
- What did management do to show commitment?
- Did management keep the data safe?

We Tried Inspections and They Didn't Work - 2

- Was the project not ready for inspections?
- Were inspections treated as a silver bullet in a chaotic environment?
- Were people trained?
- Was the process followed?

Deadly Sins of Inspections - 1

- Superficial commitment from management
- Not enough time in the schedule
- Resources are not allocated
- Insufficient preparation
- Wrong people assigned
- Not using the data

Deadly Sins of Inspections - 2

- Treating inspections as a rubber stamp
- Using the checklist without thinking beyond it
- Not training the inspectors
- No entry/exit criteria
- Wrong pace
- Believing all review types are the same

References

- Software Inspections by Ronald A. Radice
Tata-McGraw Hill 2003 (Chapter 11)

